图论

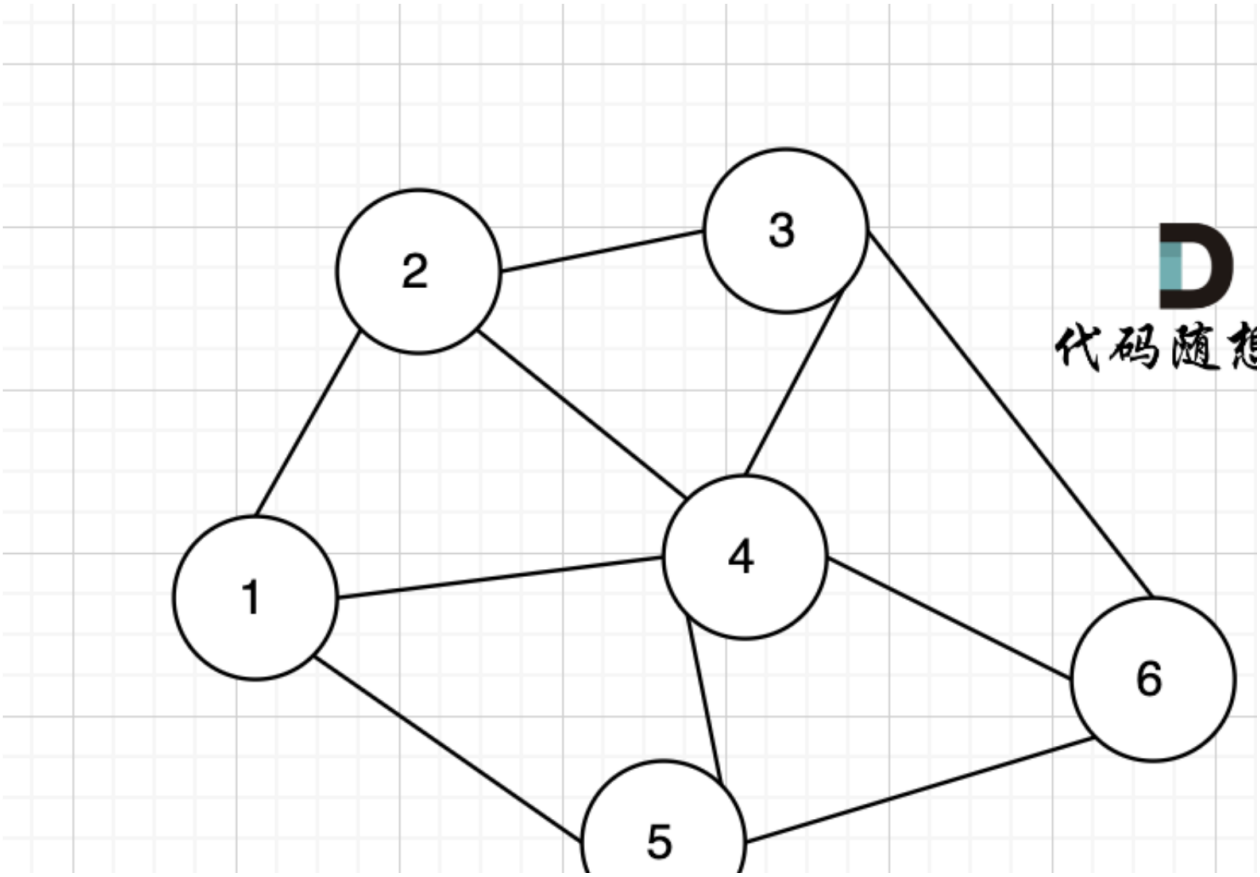
理论基础

连通性

- 无向连通图：

  #### 连通图

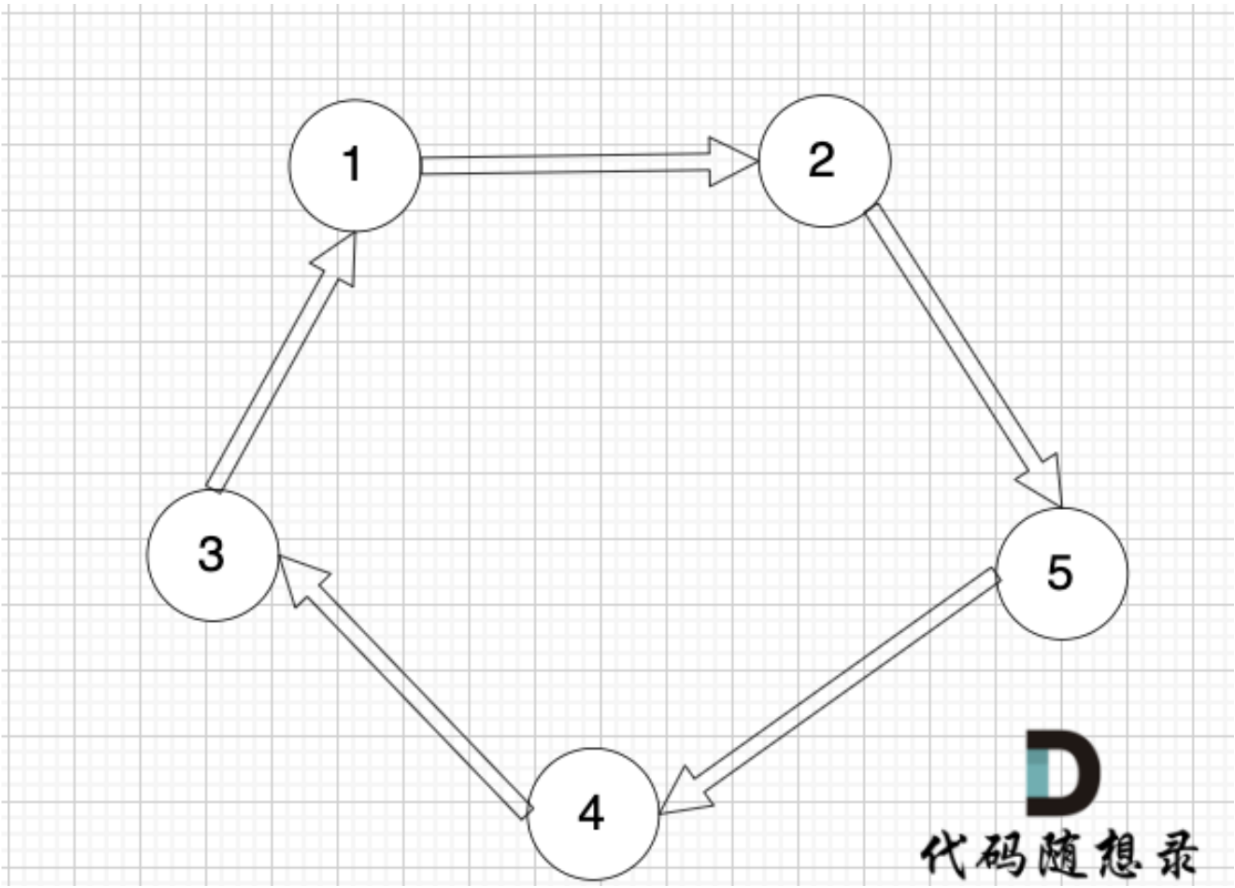
  在无向图中，任何两个节点都是可以到达的，我们称之为连通图 ，如图：



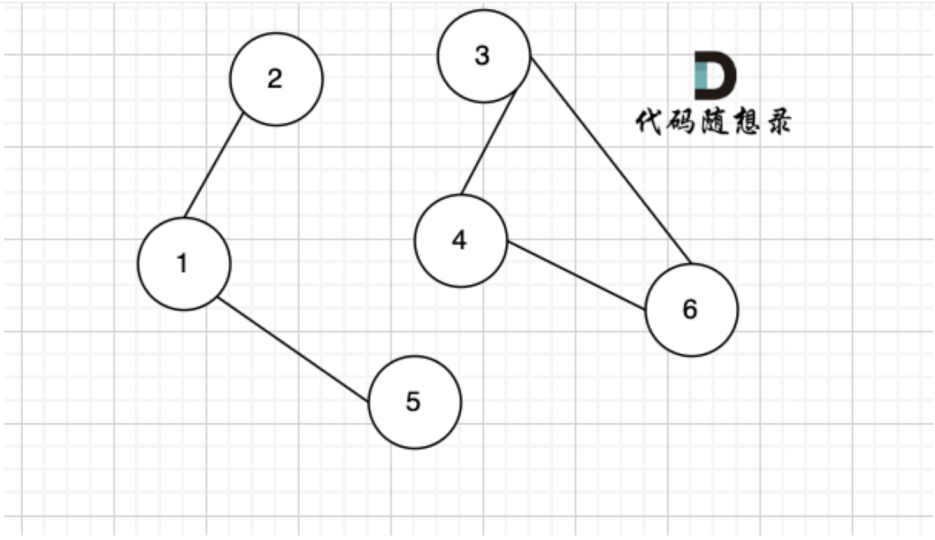
- 有向强连通图：**要相互**

  强连通图是在有向图中**任何两个节点是可以相互到达**

  下面这个有向图才是强连通图：



- 连通分量：



该无向图中 节点1、节点2、节点5 构成的子图就是 该无向图中的一个连通分量，该子图所有节点都是相互可达到

同理，节点3、节点4、节点6 构成的子图 也是该无向图中的一个连通分量。

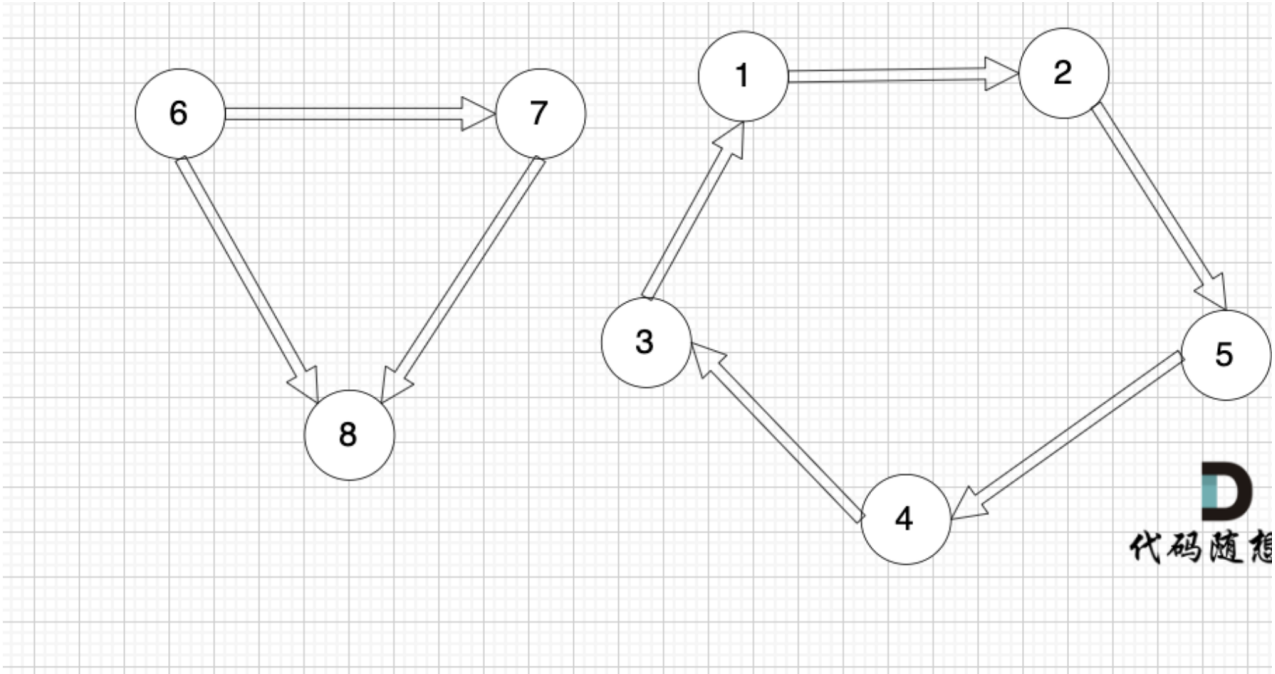
那么无向图中 节点3 、节点4 构成的子图 是该无向图的联通分量吗？

不是！

因为必须是极大联通子图才能是连通分量，所以 必须是节点3、节点4、节点6 构成的子图才是连通分量。

在图论中，连通分量是一个很重要的概念，例如岛屿问题（后面章节会有专门讲解）其实就是求连通分量。

- 强连通分量：



节点1、节点2、节点3、节点4、节点5 构成的子图是强连通分量，因为这是强连通图，也是极大图。

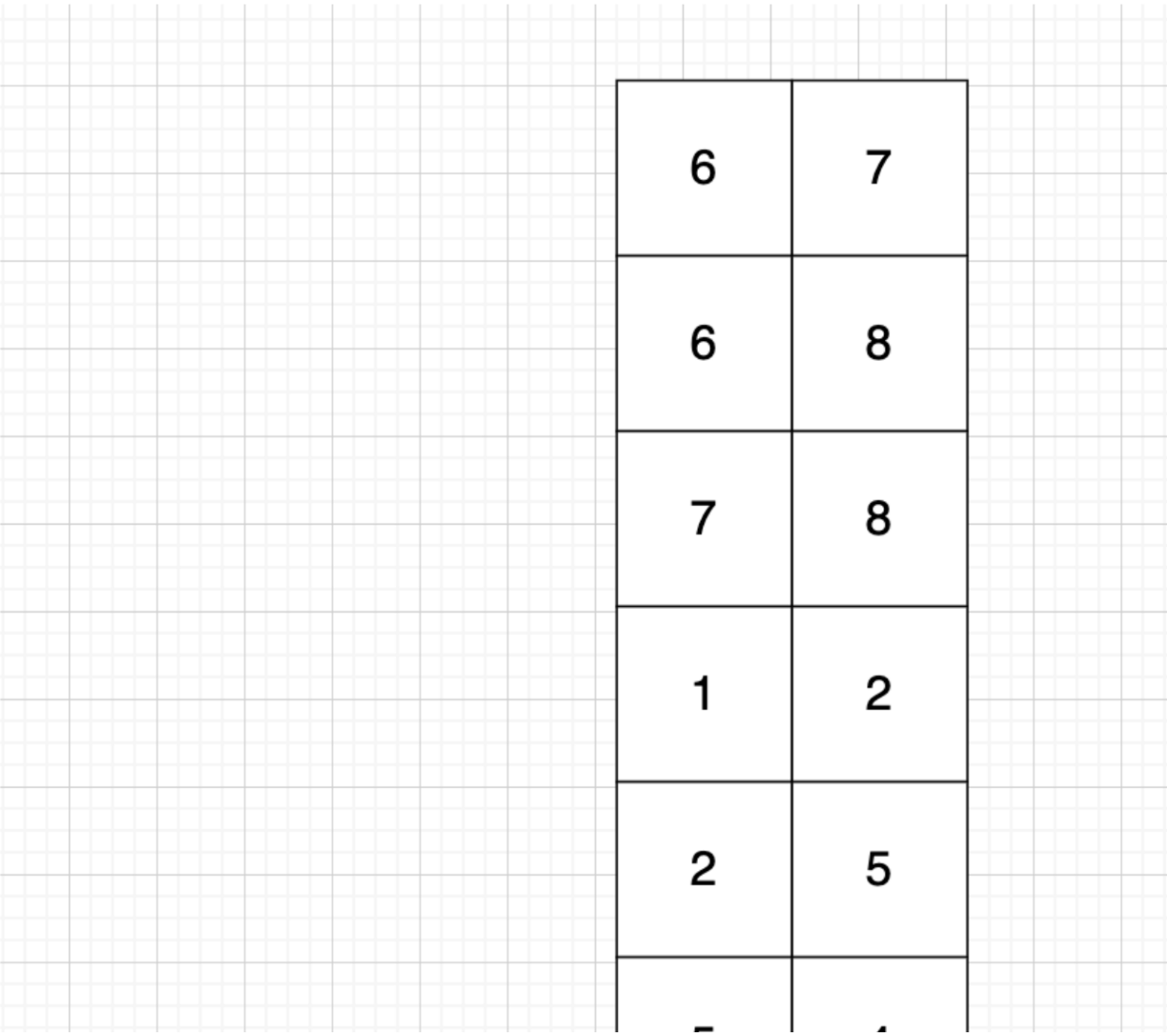
节点6、节点7、节点8 构成的子图 不是强连通分量，因为这不是强连通图，节点8 不能达到节点6。

节点1、节点2、节点5 构成的子图 也不是 强连通分量，因为这不是极大图。

图的构造

- 朴素存储：如图第一行指6指向7

  图中有8条边，我们就定义 8 * 2的数组，即有n条边就申请n * 2，这么大的数组：



- 邻接矩阵：如图指节点2指向节点5，权值为6

  在一个 n （节点数）为8 的图中，就需要申请 8 * 8 这么大的空间。

  图中有一条双向边，即：grid[2][5] = 6，grid[5][2] = 6

  这种表达方式（邻接矩阵） 在 边少，节点多的情况下，会导致申请过大的二维数组，造成空间浪费。

  而且在寻找节点连接情况的时候，需要遍历整个矩阵，即 n * n 的时间复杂度，同样造成时间浪费。

  邻接矩阵的优点：

  - 表达方式简单，易于理解
  - 检查任意两个顶点间是否存在边的操作非常快
  - 适合稠密图，在边数接近顶点数平方的图中，邻接矩阵是一种空间效率较高的表示方法。

  缺点：

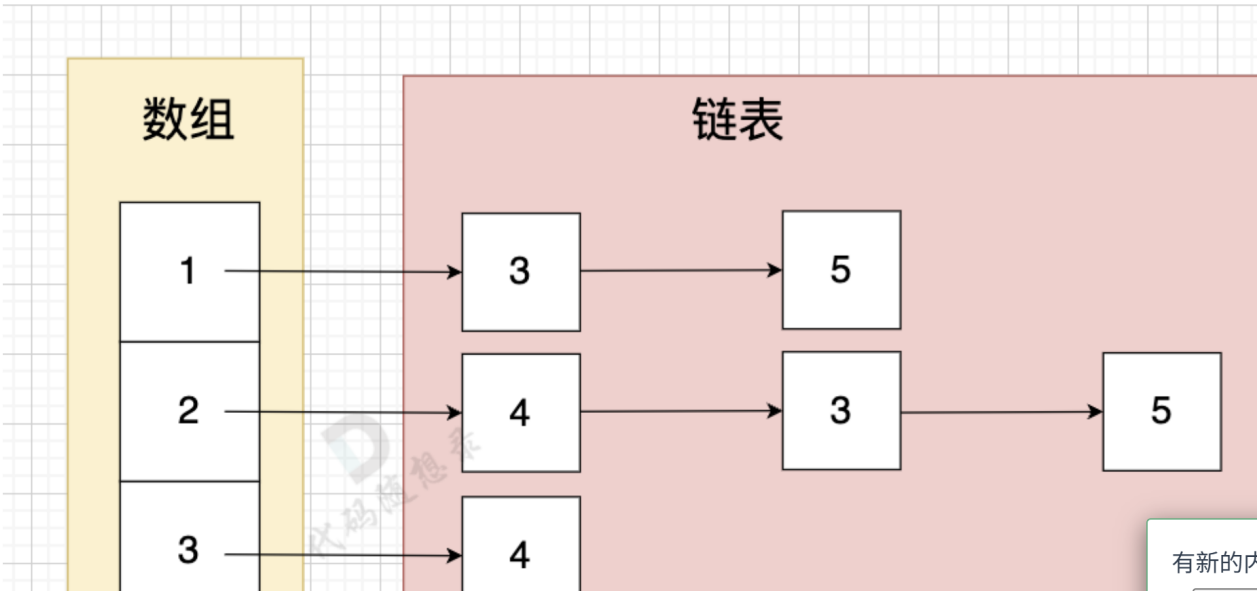
  - 遇到稀疏图，会导致申请过大的二维数组造成空间浪费 且遍历 边 的时候需要遍历整个n * n矩阵，造成时间浪费

- 邻接表：

  邻接表

  邻接表 使用 数组 + 链表的方式来表示。 邻接表是从边的数量来表示图，有多少边 才会申请对应大小的链表。

  邻接表的构造如图：



深搜

```
void dfs(参数) {
    if (终止条件) {
        存放结果;
        return;
    }

    for (选择：本节点所连接的其他节点) {
        处理节点;
        dfs(图，选择的节点); // 递归
        回溯，撤销处理结果
    }
}
```

所有可达路径

98. 所有可达路径

卡码网题目链接（ACM模式）

【题目描述】

给定一个有 n 个节点的有向无环图，节点编号从 1 到 n。请编写一个程序，找出并返回所有从节点 1 到节点 n 的路径。每条路径应以节点编号的列表形式表示。

【输入描述】

第一行包含两个整数 N，M，表示图中拥有 N 个节点，M 条边

后续 M 行，每行包含两个整数 s 和 t，表示图中的 s 节点与 t 节点中有一条路径

【输出描述】

输出所有的可达路径，路径中所有节点的后面跟一个空格，每条路径独占一行，存在多条路径，路径输出的顺序可任意。

如果不存在任何一条路径，则输出 -1。

注意输出的序列中，最后一个节点后面没有空格！ 例如正确的答案是 `1 3 5` ,而不是 `1 3 5` ， 5后面没有空格！

- 邻接矩阵

```
#include <iostream>
#include <vector>
using namespace std;
vector<vector<int>> result; // 收集符合条件的路径
vector<int> path; // 1节点到终点的路径

void dfs (const vector<vector<int>>& graph, int x, int n) {
    // 当前遍历的节点x 到达节点n
    if (x == n) { // 找到符合条件的一条路径
        result.push_back(path);
        return;
    }
    for (int i = 1; i <= n; i++) { // 遍历节点x链接的所有节点
        if (graph[x][i] == 1) { // 找到 x链接的节点
            path.push_back(i); // 遍历到的节点加入到路径中来
            dfs(graph, i, n); // 进入下一层递归
            path.pop_back(); // 回溯，撤销本节点
        }
    }
}

int main() {
    int n, m, s, t;
    cin >> n >> m;

    // 节点编号从1到n，所以申请 n+1 这么大的数组
    vector<vector<int>> graph(n + 1, vector<int>(n + 1, 0));

    while (m--) {
```

```
        cin >> s >> t;
        // 使用邻接矩阵 表示无线图，1 表示 s 与 t 是相连的
        graph[s][t] = 1;
    }

    path.push_back(1); // 无论什么路径已经是从0节点出发
    dfs(graph, 1, n); // 开始遍历

    // 输出结果
    if (result.size() == 0) cout << -1 << endl;
    for (const vector<int> &pa : result) {
        for (int i = 0; i < pa.size() - 1; i++) {
            cout << pa[i] << " ";
        }
        cout << pa[pa.size() - 1]  << endl;
    }
}
```

- 邻接表

```
#include <iostream>
#include <vector>
#include <list>
using namespace std;

vector<vector<int>> result; // 收集符合条件的路径
vector<int> path; // 1节点到终点的路径

void dfs (const vector<list<int>>& graph, int x, int n) {

    if (x == n) { // 找到符合条件的一条路径
        result.push_back(path);
        return;
    }
    for (int i : graph[x]) { // 找到 x指向的节点
        path.push_back(i); // 遍历到的节点加入到路径中来
        dfs(graph, i, n); // 进入下一层递归
        path.pop_back(); // 回溯，撤销本节点
    }
}

int main() {
    int n, m, s, t;
    cin >> n >> m;

    // 节点编号从1到n，所以申请 n+1 这么大的数组
    vector<list<int>> graph(n + 1); // 邻接表
    while (m--) {
        cin >> s >> t;
        // 使用邻接表 ，表示 s -> t 是相连的
        graph[s].push_back(t);
```

```
    }

    path.push_back(1); // 无论什么路径已经是从0节点出发
    dfs(graph, 1, n); // 开始遍历

    // 输出结果
    if (result.size() == 0) cout << -1 << endl;
    for (const vector<int> &pa : result) {
        for (int i = 0; i < pa.size() - 1; i++) {
            cout << pa[i] << " ";
        }
        cout << pa[pa.size() - 1]  << endl;
    }
}
```

广搜

我们从图中可以看出，从start起点开始，是一圈一圈，向外搜索，方格编号1为第一步遍历的节点，方格编号2为第二步遍历的节点，第四步的时候我们找到终止点end。

正是因为BFS一圈一圈的遍历方式，所以一旦遇到终止点，那么一定是一条最短路径。

而且地图还可以有障碍，如图所示：

```
int dir[4][2] = {0, 1, 1, 0, -1, 0, 0, -1}; // 表示四个方向
// grid 是地图，也就是一个二维数组
// visited标记访问过的节点，不要重复访问
// x,y 表示开始搜索节点的下标
void bfs(vector<vector<char>>& grid, vector<vector<bool>>& visited, int x, int y)
{
    queue<pair<int, int>> que; // 定义队列
    que.push({x, y}); // 起始节点加入队列
    visited[x][y] = true; // 只要加入队列，立刻标记为访问过的节点
```

```
    while(!que.empty()) { // 开始遍历队列里的元素
        pair<int ,int> cur = que.front(); que.pop(); // 从队列取元素
        int curx = cur.first;
        int cury = cur.second; // 当前节点坐标
        for (int i = 0; i < 4; i++) { // 开始想当前节点的四个方向左右上下去遍历
            int nextx = curx + dir[i][0];
            int nexty = cury + dir[i][1]; // 获取周边四个方向的坐标
            if (nextx < 0 || nextx >= grid.size() || nexty < 0 || nexty >=
grid[0].size()) continue;  // 坐标越界了，直接跳过
            if (!visited[nextx][nexty]) { // 如果节点没被访问过
                que.push({nextx, nexty});  // 队列添加该节点为下一轮要遍历的节点
                visited[nextx][nexty] = true; // 只要加入队列立刻标记，避免重复访问
            }
        }
    }

}
```

岛屿数量

99. 岛屿数量

卡码网题目链接（ACM模式）

题目描述：

给定一个由 1（陆地）和 0（水）组成的矩阵，你需要计算岛屿的数量。岛屿由水平方向或垂直方向上相邻的陆地连接而成，并且四周都是水域。你可以假设矩阵外均被水包围。

输入描述：

第一行包含两个整数 N, M，表示矩阵的行数和列数。

后续 N 行，每行包含 M 个数字，数字为 1 或者 0。

输出描述：

输出一个整数，表示岛屿的数量。如果不存在岛屿，则输出 0。

输入示例：

- 本题思路，是用遇到一个没有遍历过的节点陆地，计数器就加一，然后把该节点陆地所能遍历到的陆地都标记上
- 在遇到标记过的陆地节点和海洋节点的时候直接跳过。 这样计数器就是最终岛屿的数量
- 深搜：

```
#include <iostream>
#include <vector>
using namespace std;
int dir[4][2] = {0, 1, 1, 0, -1, 0, 0, -1}; // 四个方向
void dfs(const vector<vector<int>>& grid, vector<vector<bool>>& visited, int x,
int y) {
```

```
    if (visited[x][y] || grid[x][y] == 0) return; // 终止条件：访问过的节点 或者 遇到海水
    visited[x][y] = true; // 标记访问过
    for (int i = 0; i < 4; i++) {
        int nextx = x + dir[i][0];
        int nexty = y + dir[i][1];
        if (nextx < 0 || nextx >= grid.size() || nexty < 0 || nexty >= grid[0].size()) continue;  // 越界了，直接跳过
        dfs(grid, visited, nextx, nexty);
    }
}

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> grid(n, vector<int>(m, 0));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> grid[i][j];
        }
    }

    vector<vector<bool>> visited(n, vector<bool>(m, false));

    int result = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (!visited[i][j] && grid[i][j] == 1) {
                result++; // 遇到没访问过的陆地，+1
                dfs(grid, visited, i, j); // 将与其链接的陆地都标记上 true
            }
        }
    }
    cout << result << endl;
}
```

- 广搜：

广度优先搜索

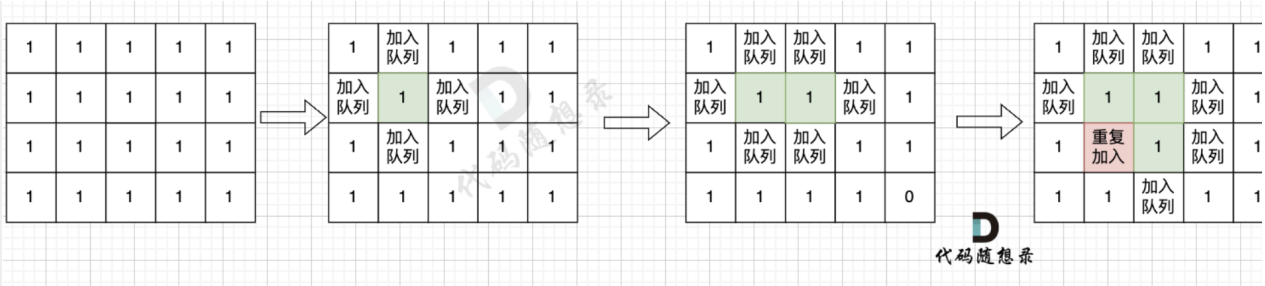
如果不熟悉广搜，建议先看 广搜理论基础。

不少同学用广搜做这道题目的时候，超时了。 这里有一个广搜中很重要的细节：

根本原因是**只要 加入队列就代表 走过，就需要标记，而不是从队列拿出来的时候再去标记走过。**

很多同学可能感觉这有区别吗？

如果从队列拿出节点，再去标记这个节点走过，就会发生下图所示的结果，会导致很多节点重复加入队列。




```
// 区别只在bfs函数
void bfs(const vector<vector<int>>& grid, vector<vector<bool>>& visited, int x,
int y) {
    queue<pair<int, int>> que;
    que.push({x, y});
    visited[x][y] = true; // 只要加入队列，立刻标记
    while(!que.empty()) {
        pair<int ,int> cur = que.front(); que.pop();
        int curx = cur.first;
        int cury = cur.second;
        for (int i = 0; i < 4; i++) {
            int nextx = curx + dir[i][0];
            int nexty = cury + dir[i][1];
            if (nextx < 0 || nextx >= grid.size() || nexty < 0 || nexty >=
grid[0].size()) continue;  // 越界了，直接跳过
            if (!visited[nextx][nexty] && grid[nextx][nexty] == 1) {
                que.push({nextx, nexty});
                visited[nextx][nexty] = true; // 只要加入队列立刻标记
            }
        }
    }
}
```